Spring 2026

Automated Disaster Settlement (A.D.S.)

System Architecture Specification

Team 6: Danial Haroon, Alan Fuentes, and Sai Harsha Venugopal

Table Of Contents

Concept

- Core Problem: Disaster verification for payout/response decisions is slow, inconsistent, and poorly auditable.

- Solution: Automate verification of real-world disaster events using public evidence to support decisions (insurance, triggers, response validation).

Functional Requirements

FR 1.1.1: Disaster Evidence Collection

FR 1.1.2: Evidence Standardization

FR 1.1.3: Trigger Condition Evaluation

FR 1.1.4: Confidence Score Generation

FR 1.1.5: Evidence Report Generation

FR 1.1.6: Search Verified Events

FR 1.1.7: Map and Timeline View

Non-Functional Requirements

NFR 1.2.1: Auditability and Traceability

NFR 1.2.2: Consistent Decision Output

NFR 1.2.3: Reliability Under Partial Data

NFR 1.2.4: Performance

NFR 1.2.5: Security

NFR 1.2.6: Usability

System Constraints

Tool Constraints

- → Python Development Environment
- → Version Control System

Language Constraints

- → Programming Language Restriction
- → Requirements Documentation Language

Platform Constraints

- → Cross-Platform Compatibility
- → Web Browser Compatibility

Hardware Constraints

- → Standardized Computing Hardware

System Constraints Continued

Network Constraints

→ Internet Connectivity Requirement

Deployment Constraints

→ Secure API Connectivity

Transition and Support Constraints

→ System Documentation Availability

Budget and Schedule Constraints

→ Open-Source Technology Use

Evolutionary Requirements

01 Future FR 1: Real-time disaster data integration

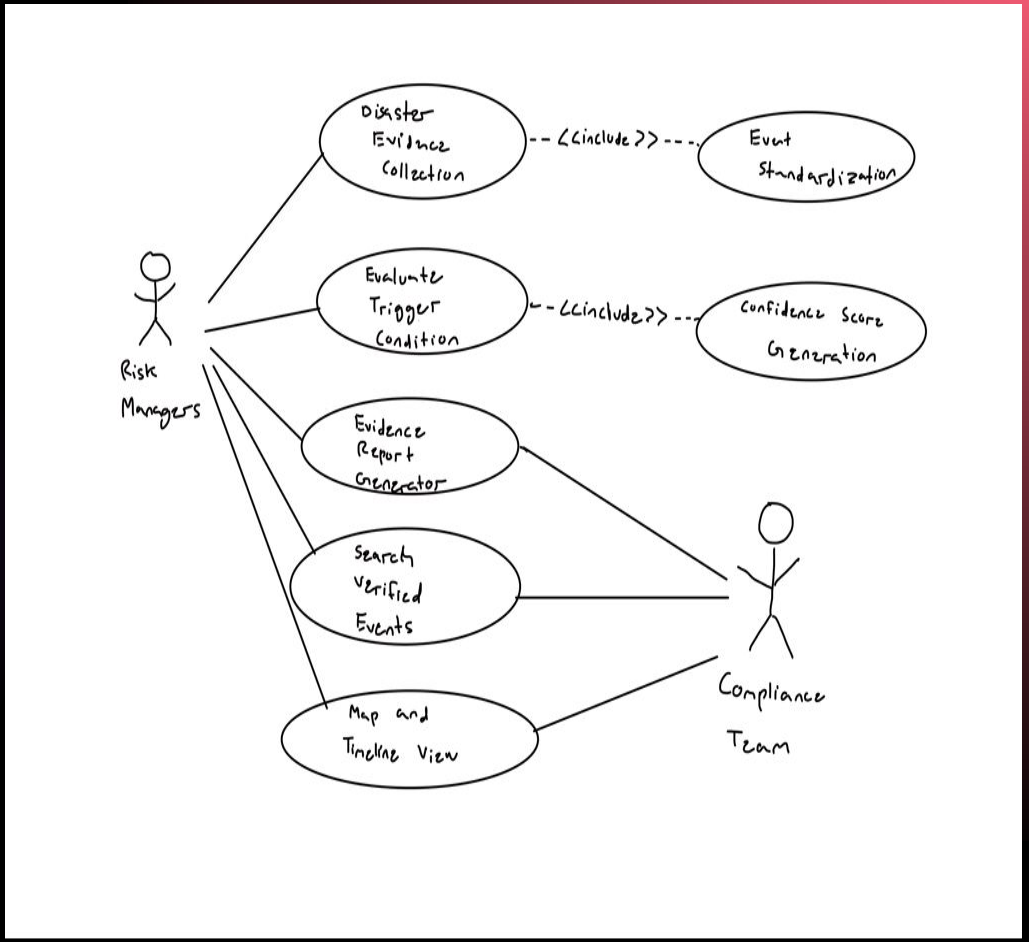
02 Future FR 2: Automated alert/notification system

03 Future NFR 1: Scalability for large-scale data processing

04 Future NFR 2: High availability and system reliability

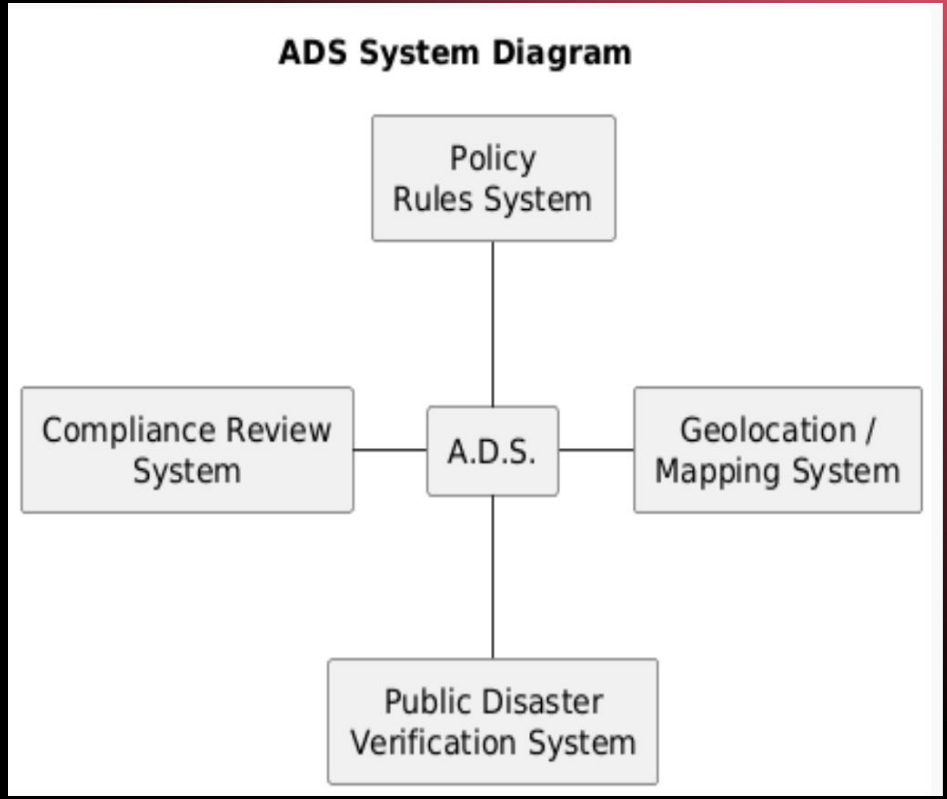
Use Cases and Use Case Diagram

- Actors/Users
  - Risk Managers
  - Compliance Team
- Use Cases/Scenarios
  - Disaster Evidence Collection
  - Event Standardization
  - Evaluate Trigger Condition
  - Confidence Score Generation
  - Evidence Report Generation
  - Search Verified Events
  - Map and Timeline View

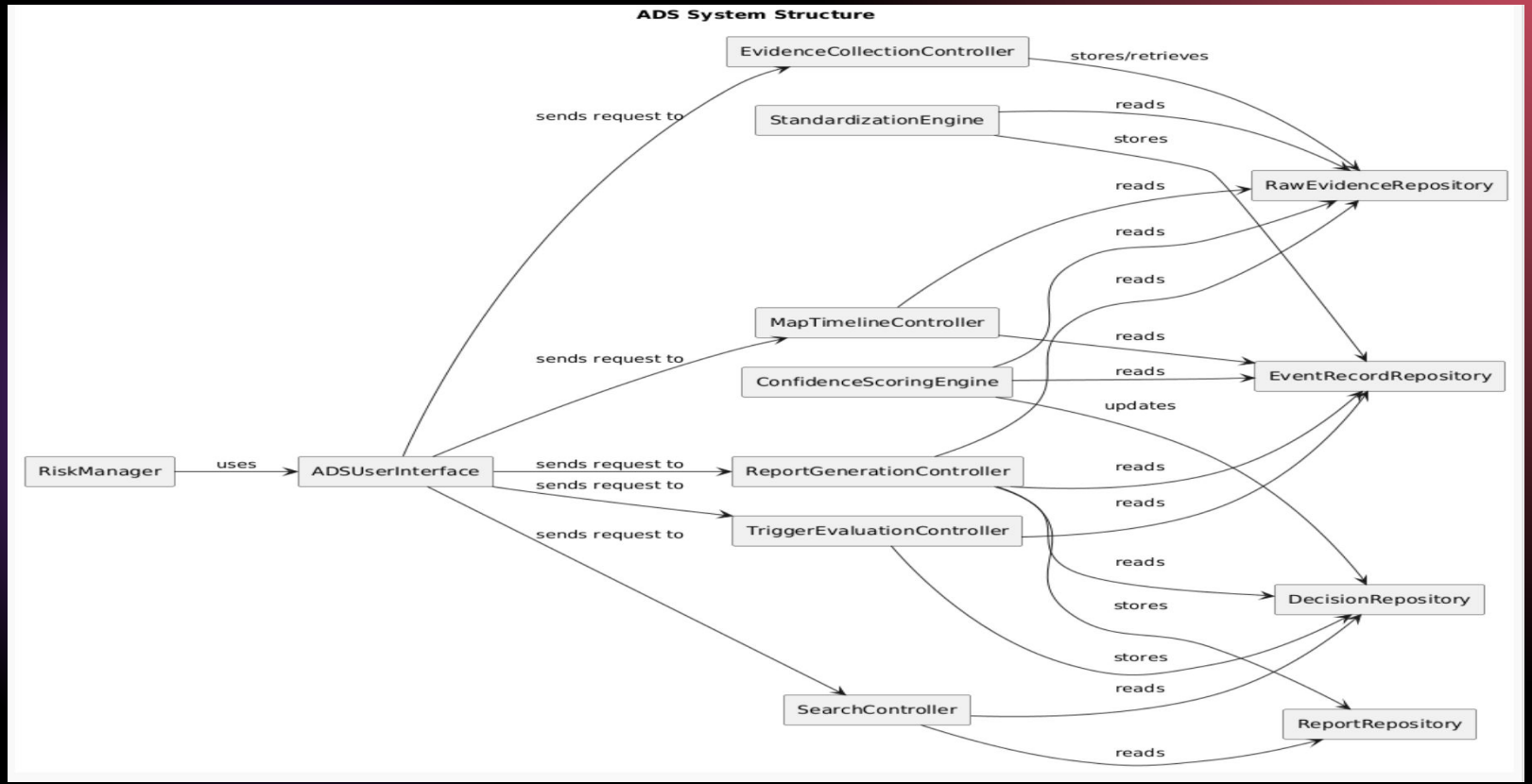


System Model: System Diagram

- Systems
  - Policy Rule System
  - Compliance Review System
  - Public Disaster Verification System
  - Geolocation/Mapping System

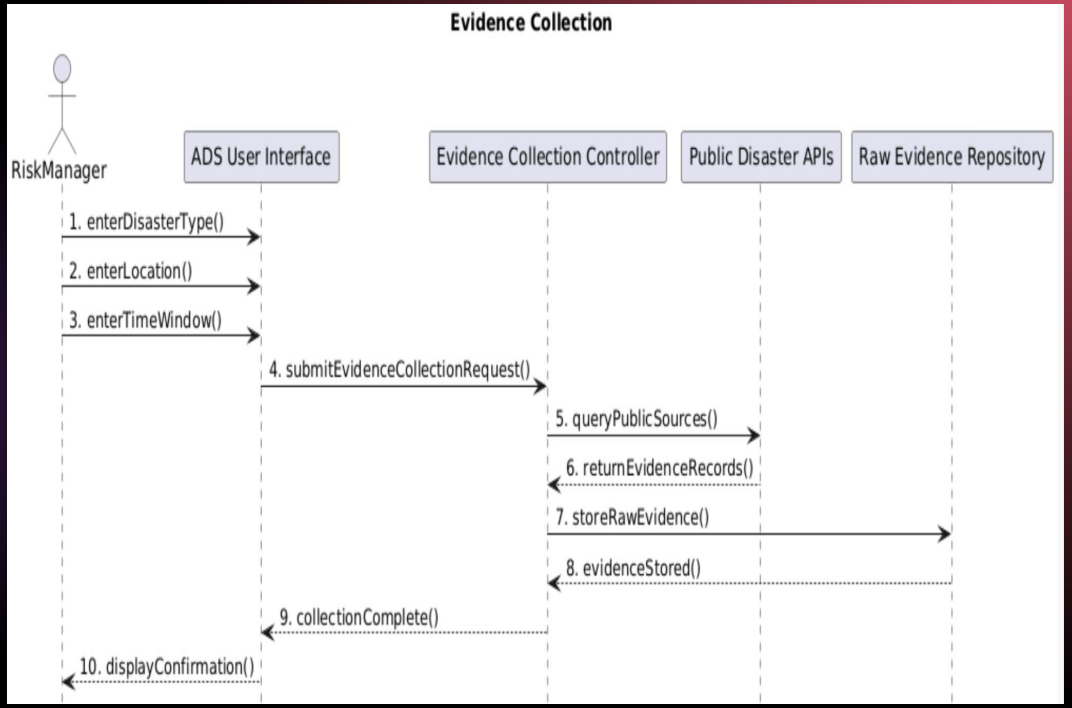


System Model: A look into the Structure of A.D.S.



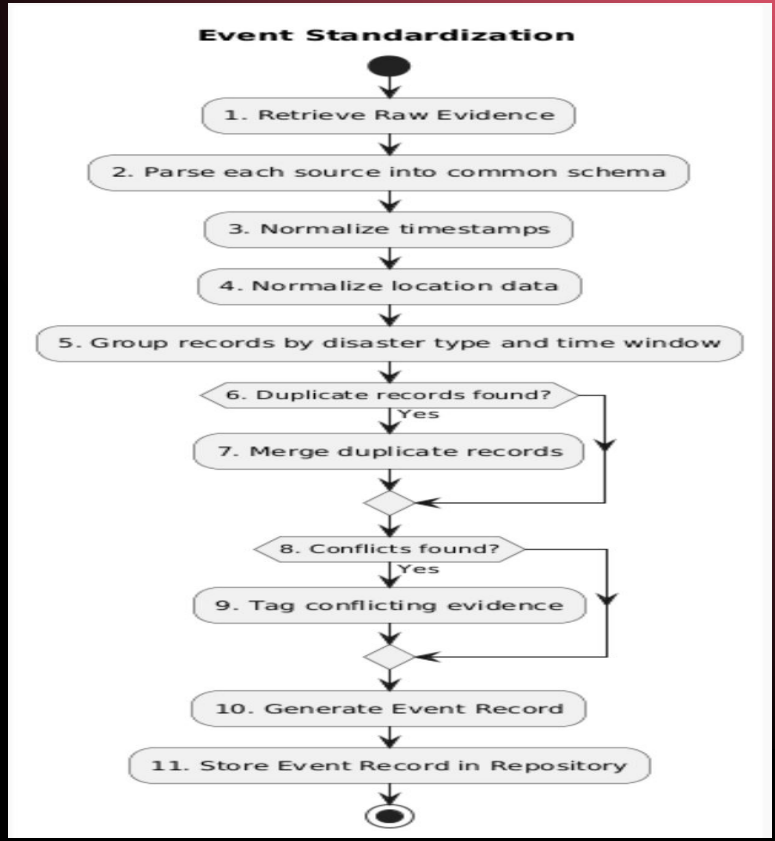
System Model: Sequence Diagram for Disaster Evidence Collection

- Objects Involved
  - Risk Manager
  - ADS User Interface
  - Evidence Collection Controller
  - Public Disaster APIs
  - Raw Evidence Repository



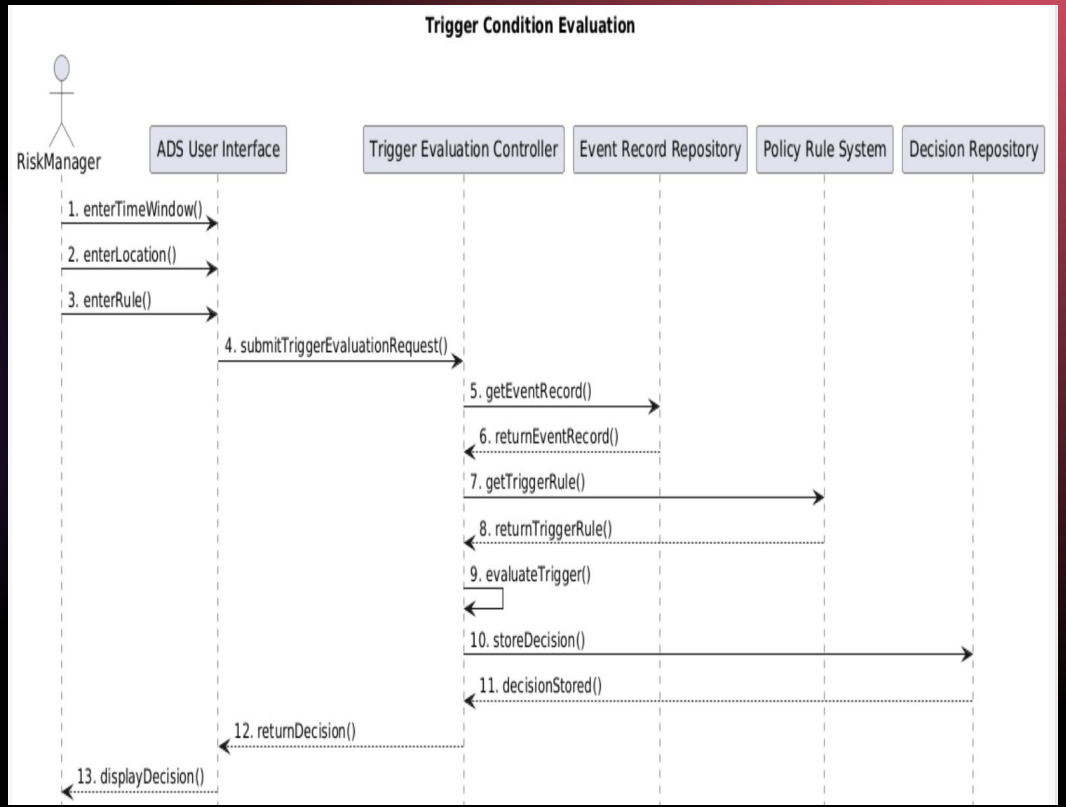
System Model: Activity Diagram for Event Standardization

- Process Overview
  - Normalize raw evidence data
  - Group related records
  - Handle duplicates and conflicts
  - Generate structured event record



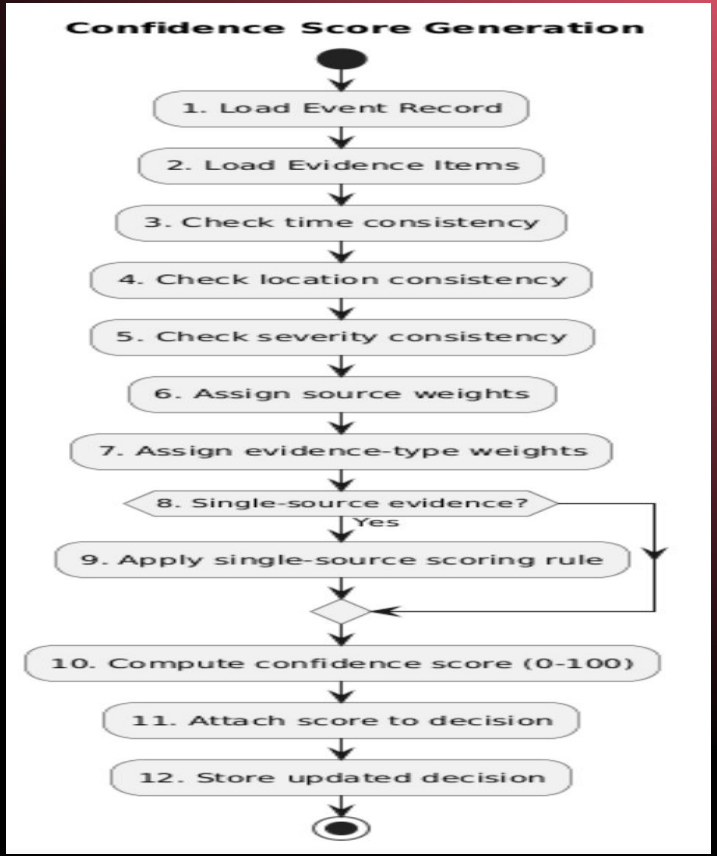
System Model: Sequence Diagram for Trigger Condition Evaluation

- Objects Involved
  - Risk Manager
  - ADS User Interface
  - Trigger Evaluation Controller
  - Event Record Repository
  - Policy Rule System
  - Decision Repository



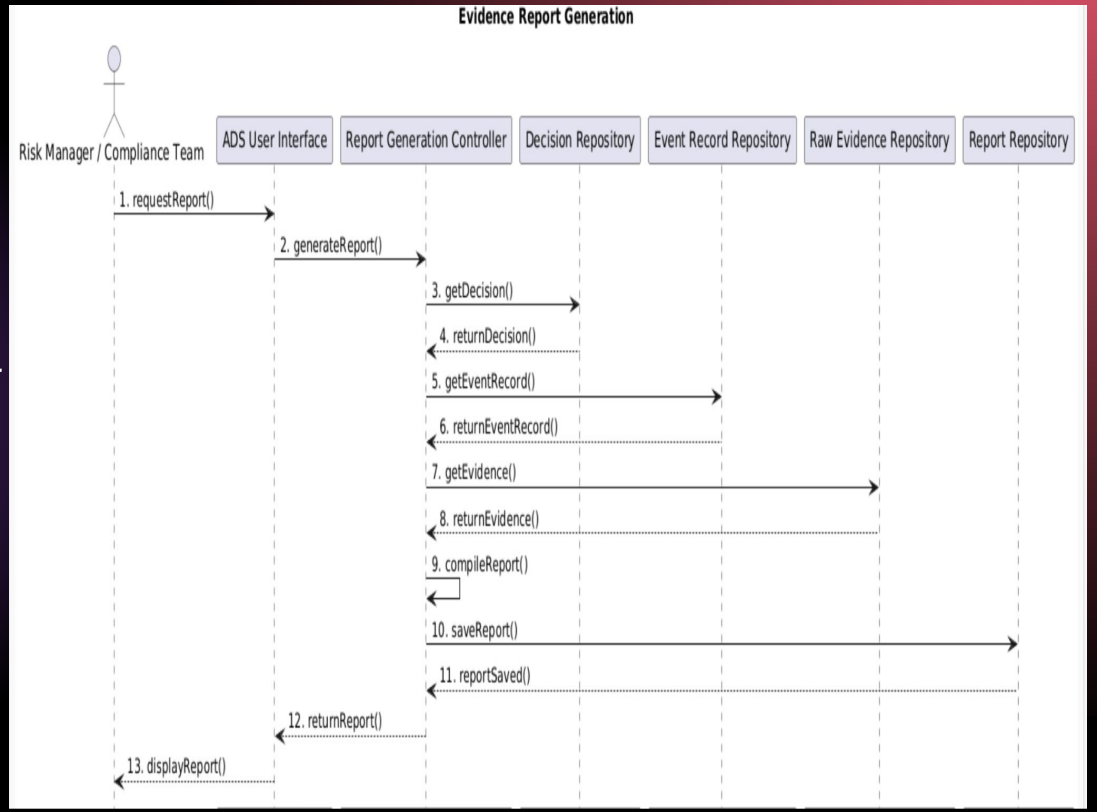
System Model: Activity Diagram for Confidence Score Generation

- Process Overview
  - Evaluate evidence consistency
  - Apply scoring logic
  - Compute confidence score
  - Attach score to decision



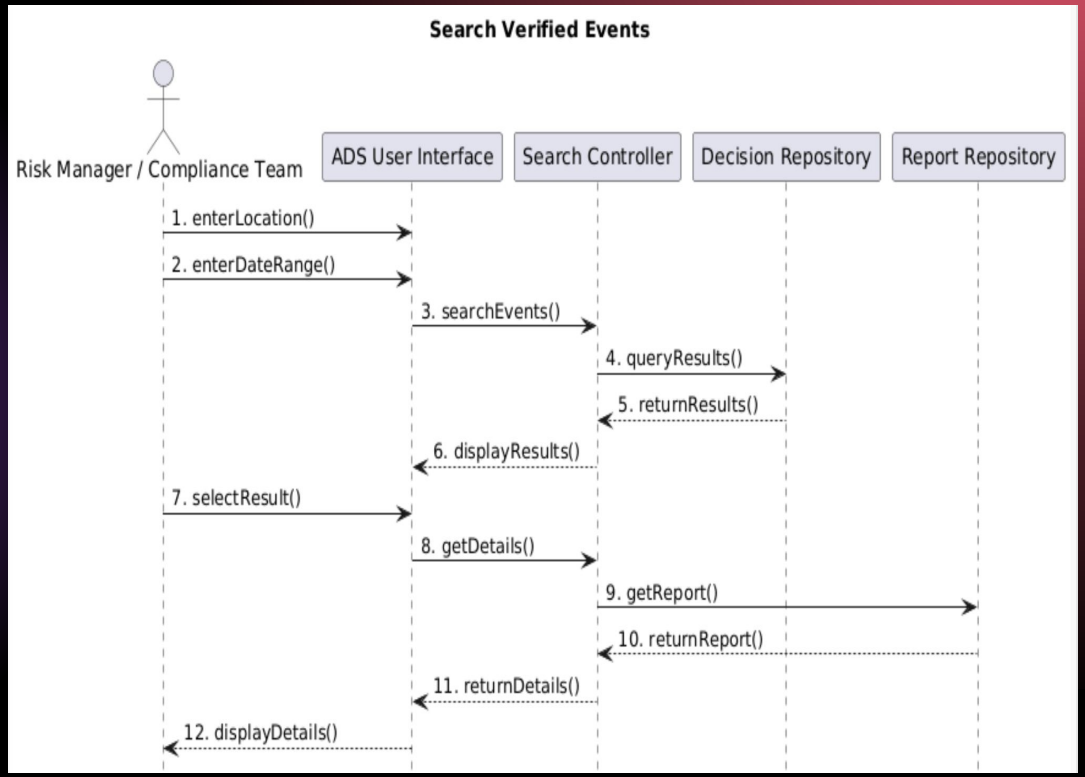
System Model: Sequence Diagram for Evidence Report Generation

- Objects Involved
  - Risk Manager/Compliance Team
  - ADS User Interface
  - Report Generation Controller
  - Decision Repository
  - Event Record Repository
  - Raw Evidence Repository
  - Report Repository



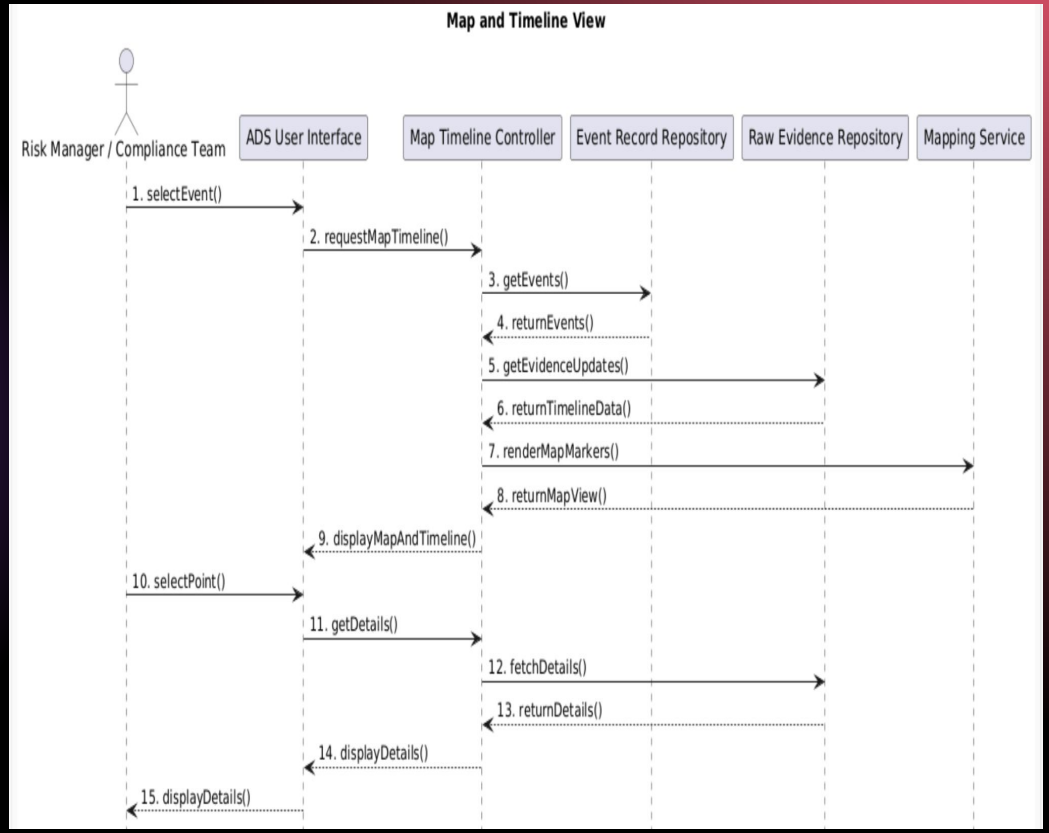
System Model: Sequence Diagram for Search Verified Events

- Objects Involved
  - Risk Manager/Compliance Team
  - ADS User Interface
  - Search Controller
  - Decision Repository
  - Report Repository



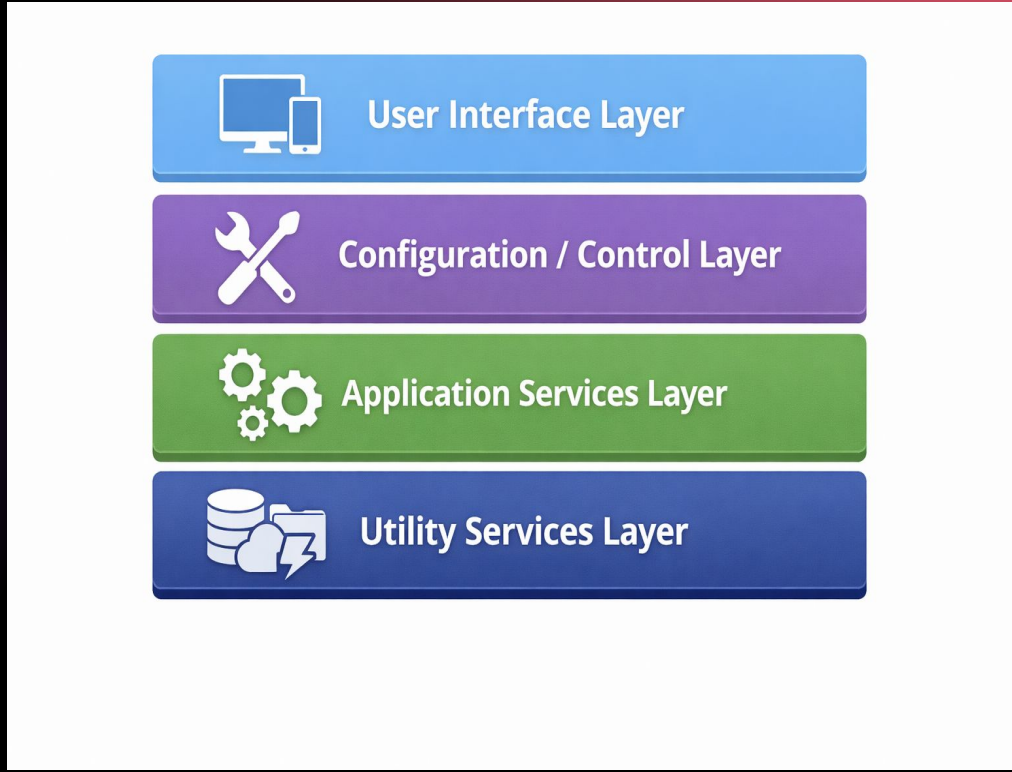
System Model: Sequence Diagram for Map and Timeline View

- Objects Involved
  - Risk Manager/Compliance Team
  - ADS User Interface
  - Map Timeline Controller
  - Event Record Repository
  - Raw Evidence Repository
  - Mapping Service



System Architecture Design

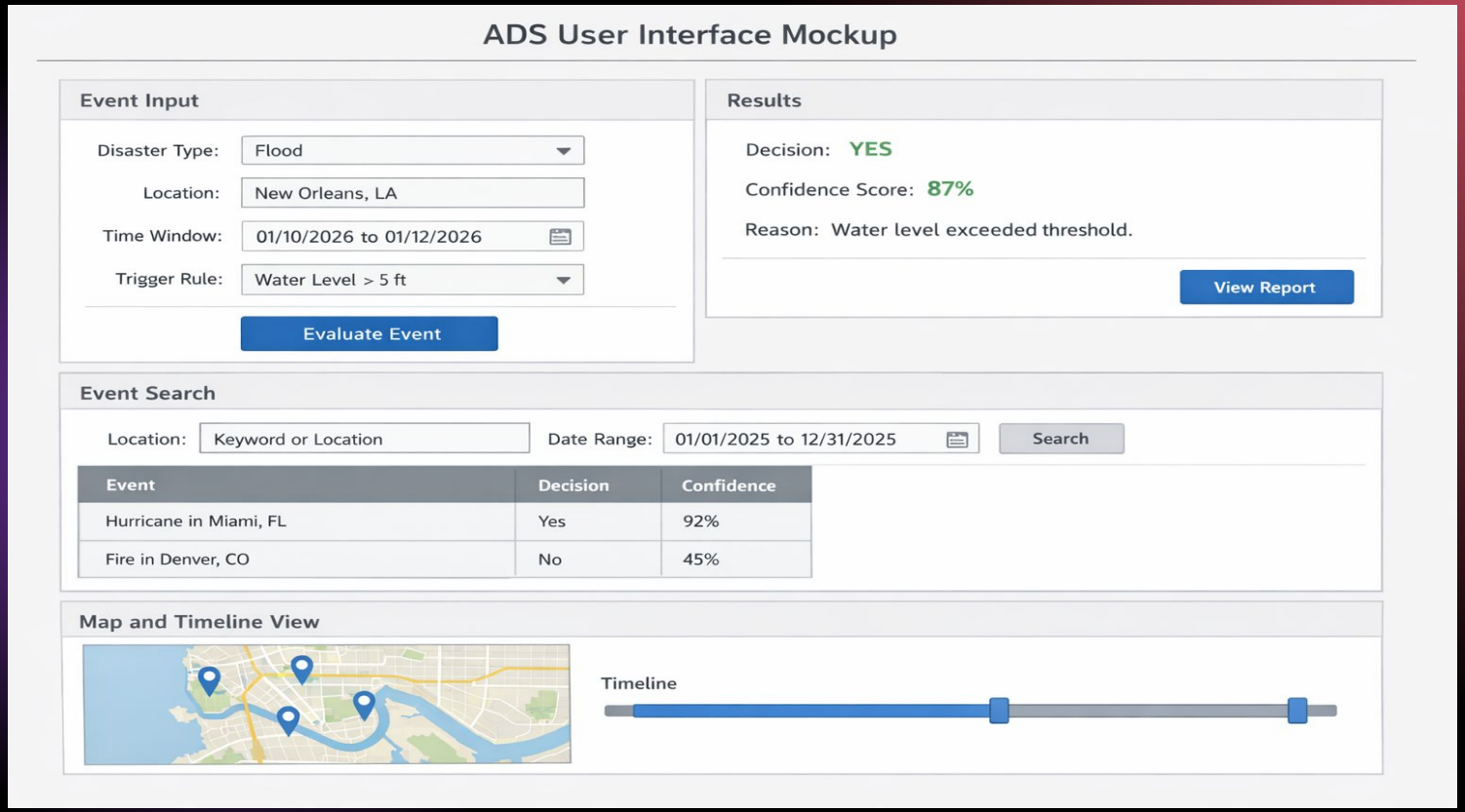
- First Layer: User Interface
- Second Layer: Configuration/Control Layer
- Third Layer: Application Services Layer
- Fourth Layer: Utility Services Layer



Design Patterns

- MVC (Model–View–Controller)
  - Separates user interface, control logic, and data model
  - View: ADS User Interface
  - Controller: System controllers
  - Model: Core data objects (EventRecord, EvidenceItem, etc.)
- Repository Pattern
  - Manages data storage and retrieval separately from logic
  - Used for Raw Evidence, Event Records, Decisions, and Reports

Mockup User Interface



Conclusion

- Designed a structured system for automated disaster verification
- Used layered architecture and design patterns (MVC, Repository)
- Modeled system behavior with UML diagrams
- Supports accurate, consistent, and auditable decision-making

References

- Images on slide 3,4,5,19,21 were generated by ChatGPT.

Q&A

QR Code For Voting